Business Requirements Document

Prometheus Financial Core — Loan Ledger Edition

Consolidated Business Requirements Document

Prepared by: Business Analyst Agent
Date: August 16, 2026
Version: 1.0 (Draft — Scaffold)

Status: Illustrative Scaffold, Not Validated Requirements.
No raw business input was supplied for this document. All six epics below were generated by the Business Analyst Agent as a demonstration of scope, structure, and ledger-integrity discipline. Every numeric threshold, fee rule, and business term referenced (rates, grace periods, non-accrual thresholds, etc.) is illustrative and must be validated against actual product terms before use in design or development.

Scope & System Context

This document covers requirements for the Prometheus Financial Core (Loan Ledger Edition), whose sole purpose is to serve as the ledger of record for a personal loan product. Scope is deliberately narrow — no deposits, cards, or other retail products.

In-Scope Modules

- Party/CIF
- CRM
- General Ledger & Posting Engine
- Loan Account Subledger
- Payment Execution
- Batch/EOD & Reconciliation

External Contracts (Adapters)

The system exposes exactly five contracts to the outside world, consumed by the Personal Loan Solution's domain services. It has no direct customer-facing channel.

- PartyAdapter
- CRMAdapter
- AccountAdapter
- GLPostingAdapter
- PaymentAdapter

Governing Principles

- Every balance-affecting requirement must ultimately resolve to a **balanced, immutable journal entry** — there is no such thing as a requirement that "just updates a balance."
- **CRM requirements are the one exception**: CRM requirements never touch a balance at all — if one seems to, it belongs to a different module.
- Any requirement implying a direct balance mutation outside the Posting Engine, a new payment rail, or scope beyond the six modules is **explicitly flagged for architecture review** rather than silently scoped in.

Document Coverage

8 epics · 60 user stories · 61 traceable requirements with Gherkin acceptance criteria · 24 architecture-review flags.

Table of Contents

2. 3. Loan Disbursement Processing [CB-DISB]
3. 4. Daily Interest Accrual [CB-ACCR]
4. 5. Loan Repayment Processing [CB-REPAY]
5. 6. Delinquency & Past-Due Fee Assessment [CB-DELINQ]
6. 7. Early Settlement / Loan Payoff [CB-PAYOFF]
7. 8. Loan Charge-off & Write-off [CB-CHARGEOFF]
8. 9. Loan Modification (Rate/Term Change) [CB-MOD]

1 Loan Account Booking [CB-BOOK]

> **Goal:** Enable the system to create a loan account in the Loan Account Subledger from an already-approved, underwritten loan decision — establishing the Party/CIF link and immutable contractual terms (principal, rate, term, day-count convention) in “Approved” (undisbursed) status — so that a booked account exists as the single source of truth for all downstream disbursement, repayment, and accrual processing. Underwriting and credit decisioning are explicitly out of scope; this epic begins only once an approval decision is received.

User Stories

1. **1.** As an Operations Analyst, I want an approved loan decision to create a loan account automatically, so that it's ready for disbursement without manual re-entry of terms.
2. **2.** As the System, I want the loan account linked to a validated Party/CIF, so that no loan account can exist without an identified, active party.
3. **3.** As the System, I want the loan account booked with immutable contractual terms, so that all future calculations (accrual, waterfall, payoff) reference a single unchanging source of truth.
4. **4.** As an Operations Analyst, I want booking to be idempotent per approved application, so that a duplicate or retried booking request never creates two loan accounts for the same approval.
5. **5.** As a Customer Service Rep, I want the account-opening event logged as a CRM interaction, so that customer contact history reflects the new loan without exposing ledger detail.
6. **6.** As a Compliance Officer, I want a booked account to carry no balance until disbursement, so that no income or receivable is recognized on a loan that hasn't been funded.
7. **7.** As the Batch/EOD process, I want newly booked accounts reconciled against approvals received that day, so that no approval goes un-booked past end-of-day.

Acceptance Criteria & Traceability

`REQ-CB-BOOK-001` — Create Loan Account from Approved Decision

Given an approval decision received from the origination/underwriting system, containing party reference, principal amount, rate, term, and day-count convention

When the loan account is booked via AccountAdapter

Then

- a loan account is created in “Approved” (undisbursed) status
- the account is not created if any required term (principal, rate, term, day-count) is missing or malformed — it is rejected to an exception queue instead

Note: *Underwriting/decisioning is out of scope — this requirement only ingests an already-made decision.*

`REQ-CB-BOOK-002` — Party/CIF Linkage at Booking

Given an approval decision referencing a party

When the loan account is booked

Then

- the system validates the party exists and is "Active" via PartyAdapter before booking completes
- booking is rejected with a specific reason code if the party is not found, inactive, or unverified

Regulatory constraint: *Same KYC-currency principle as disbursement party validation — a party check at booking, re-checked again at disbursement, since time may pass between the two.*

`REQ-CB-BOOK-003` — Immutable Contractual Terms

Given a loan account booked with principal, rate, term, and day-count convention

When the account is created

Then

- these contractual terms are recorded as immutable for the life of the account
- any subsequent change to terms (e.g., a rate modification) is recorded as a new versioned term set with an effective date, never an overwrite of the original

Ledger-integrity constraint: *Immutability of contractual terms — this is the "source of truth" that daily accrual and repayment-waterfall calculations depend on; if terms could be silently edited, every downstream calculation becomes unauditable.*

`REQ-CB-BOOK-004` — Idempotent Booking

Given an approval decision with a unique approval reference ID

When a booking request is received more than once for the same approval reference ID (e.g., due to an upstream retry)

Then

- only the first occurrence creates a loan account

- subsequent occurrences return the original booked account reference without creating a duplicate account

Ledger-integrity constraint: *Idempotency keyed on approval reference ID — prevents duplicate-account defects.*

`REQ-CB-BOOK-005` — CRM Logging of Account Opening

Given a successfully booked loan account

When booking completes

Then

- a CRM interaction record is created via CRMAdapter noting the booking date and loan account reference

- the CRM record contains no balance, term, or GL account detail

Note: *CRM exception applies — reference-only, no balance data.*

`REQ-CB-BOOK-006` — Zero Balance at Booking

Given a newly booked loan account in “Approved” status

When the account is created

Then

- no journal entry is posted and the account carries a zero receivable/interest balance

- any balance first appears only as a result of the disbursement posting

Regulatory / Ledger-integrity constraint: *No income or receivable recognition prior to funding — booking is a status/terms record only, not a balance event.*

`REQ-CB-BOOK-007` — EOD Reconciliation of Approvals to Bookings

Given all approval decisions received during business day D

When the Batch/EOD process runs

Then

- every approval decision has a corresponding booked loan account, or an exception record explaining why it does not
- no approval remains un-booked and unexplained past EOD close

Ledger-integrity constraint: *Detective control — completeness-check pattern applied to the booking step.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
AR-10	A hypothetical ask to pre-load an expected receivable balance at booking so pipeline reporting shows anticipated volume before funding	Direct balance mutation outside the Posting Engine. No balance may appear without a posted entry, including for reporting-convenience reasons.
AR-11	A hypothetical ask to collect an origination/booking fee via a new payment rail at account opening	New payment rail. Any fee collection mechanism not already integrated in PaymentAdapter requires architecture review before being scoped as a requirement.
AR-12	A hypothetical ask to auto-issue a linked debit card to the borrower at booking for future repayments	Scope beyond the six modules. Cards are explicitly out of scope per the system's charter.

Open Questions

- Can a booked (“Approved”) account expire if disbursement doesn't occur within N days, and if so, does that transition require its own requirement (auto-expiry)?
- Is term versioning actually in scope for this epic, or does it belong to a separate “Loan Modification” epic — since a rate change post-booking is arguably its own capability with its own approval workflow?

2 Loan Disbursement Processing [CB-DISB]

> **Goal:** Enable the Personal Loan Solution, via the AccountAdapter and GLPostingAdapter contracts, to move an approved loan from “booked” to “disbursed” status by executing a single, balanced, immutable journal entry that funds the loan principal, initiates the corresponding outbound payment to the borrower's designated account, and leaves a complete, reconciliable audit trail — so that Operations can fund approved loans same-day with zero unbalanced or orphaned ledger entries.

User Stories

1. **1.** As a Loan Officer, I want to initiate a disbursement request against an approved loan account, so that the borrower can receive funds without manual ledger entry.
2. **2.** As the System, I want to validate the borrower's Party/CIF status before releasing funds, so that disbursements are never made against a blocked, closed, or unverified party.
3. **3.** As the Posting Engine, I want to generate a balanced GL journal entry for every disbursement, so that the loan subledger and GL remain in agreement at all times.
4. **4.** As an Operations Analyst, I want the disbursement payment to be executed exactly once even if the request is retried, so that borrowers are never double-funded.
5. **5.** As a Customer Service Rep, I want the disbursement event logged as a CRM interaction, so that future customer contacts have visibility into the disbursement without exposing raw ledger data.
6. **6.** As a Batch/EOD process, I want all disbursements posted during the day reconciled against GL postings at end-of-day, so that any mismatch is caught before the next business day opens.
7. **7.** As an Operations Analyst, I want a returned/failed disbursement payment to trigger a reversal, so that the ledger reflects reality without altering the original entry.

Acceptance Criteria & Traceability

`REQ-CB-DISB-001` — Initiate Disbursement Request

Given a loan account in “Approved” status with an undisbursed principal balance, and the requesting user holds disbursement-initiation entitlement

When the Loan Officer submits a disbursement request via AccountAdapter

Then

- the request is accepted and placed in “Pending Disbursement” state
- no balance is altered until the Posting Engine confirms a balanced entry

Ledger-integrity constraint: *No balance mutation occurs at request time — state transition only. Actual funding happens exclusively via the balanced GL posting.*

`REQ-CB-DISB-002` — Party/CIF Validation Prior to Disbursement

Given a pending disbursement request

When the system checks the associated Party/CIF record via PartyAdapter

Then

- disbursement proceeds only if the party status is “Active” and KYC is “Verified”
- the request is rejected with a specific reason code if the party is blocked, closed, or unverified

Regulatory constraint: *KYC verification must be current per AML/KYC policy at time of disbursement, not merely at origination.*

`REQ-CB-DISB-003` — Balanced GL Posting for Disbursement

Given a validated disbursement request for principal amount P

When the Posting Engine generates the funding journal entry

Then

- the entry debits Loan Receivable and credits Cash/Nostro (or equivalent) for exactly P
- the sum of debits equals the sum of credits in the same entry
- the entry is written as immutable — corrections require an offsetting entry, never an edit
- the loan subledger balance updates only as a result of this posted entry

Ledger-integrity constraint: *Balanced entry (explicit debit/credit equality) and immutability (no in-place edits, ever).*

`REQ-CB-DISB-004` — Idempotent Payment Execution

Given a confirmed GL posting for a disbursement

When the PaymentAdapter is invoked to release funds to the borrower's designated account

Then

- the payment is executed exactly once per unique disbursement ID
- a retried or duplicate request with the same disbursement ID returns the original result without creating a second payment or a second journal entry

Ledger-integrity constraint: *Idempotency keyed on disbursement ID, enforced at the PaymentAdapter boundary.*

`REQ-CB-DISB-005` — CRM Logging of Disbursement Event

Given a successfully posted and paid disbursement

When the event completes

Then

- a CRM interaction record is created via CRMAdapter noting the disbursement date and loan account reference
- the CRM record contains no balance, GL account, or payment rail detail

Note: *Per scope rules, this CRM requirement never touches a balance — it is reference-only, consistent with CRM's exception status.*

`REQ-CB-DISB-006` — EOD Reconciliation of Disbursements

Given all disbursements posted during the business day

When the Batch/EOD process runs

Then

- the total of disbursement journal entries in the GL equals the total of confirmed payments executed via PaymentAdapter
- any variance is surfaced as an exception item before the ledger is closed for the day
- no disbursement remains in “Pending Disbursement” state past EOD without an exception raised

Ledger-integrity constraint: *Reconciliation is a detective control, not a corrective one — it cannot itself alter balances, only flag exceptions.*

`REQ-CB-DISB-007` — Reversal of Failed/Returned Disbursement

Given a disbursement payment that is returned or fails post-posting

When Operations confirms the failure

Then

- a new, separate reversing journal entry is posted (credit Loan Receivable, debit Cash/Nostro) for the original amount
- the original journal entry remains unaltered and fully traceable
- the loan account status reverts to “Approved” (undisbursed) only after the reversal entry is confirmed

Ledger-integrity constraint: *Immutability — reversal via new entry only, original entry never touched.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
AR-1	A hypothetical variant where Ops requests a “quick balance correction” tool to directly adjust the loan subledger without a journal entry	Direct balance mutation outside the Posting Engine. No requirement may update a balance without resolving to a balanced entry through GLPostingAdapter — this must not be silently scoped in.
AR-2	A hypothetical ask for disbursement via a new instant-payment/wallet rail not currently integrated in PaymentAdapter	New payment rail. Adding a rail is an adapter/architecture change, not a BA-level requirement — needs explicit review before scoping.
AR-3	A hypothetical ask to let disbursement also trigger opening of a linked savings/deposit account for the borrower	Scope beyond the six modules. Deposit products are explicitly out of scope for this system; this belongs to a different solution entirely.

Open Questions

- Is partial/staged disbursement in scope, or is it always full principal in one entry?
- Does the reversal requirement need a distinct regulatory hold period before the loan can be re-disbursed?

3 Daily Interest Accrual [CB-ACCR]

> **Goal:** Enable the Batch/EOD process to calculate and post accrued interest for every active loan account once per business day, using the loan's contractual rate, day-count convention, and start-of-day outstanding principal, resulting in a balanced, immutable journal entry per loan per day — so that accrued interest is always current for repayment waterfalls, payoff quotes, and financial reporting, without waiting for cash collection to recognize income.

User Stories

1. **1.** As the Batch/EOD process, I want interest accrued on every active loan account each business day, so that accrued interest is current for repayment allocation and reporting.
2. **2.** As the Posting Engine, I want each day's accrual posted as a balanced journal entry, so that the GL and loan subledger reflect accrued interest without waiting for cash collection.
3. **3.** As an Operations Analyst, I want accrual to run exactly once per loan per business day even if the batch is retried, so that interest is never double-accrued.
4. **4.** As the System, I want accrual calculated from the loan's contractual rate, day-count convention, and start-of-day outstanding principal, so that the calculation is consistent, deterministic, and auditable.
5. **5.** As a Compliance Officer, I want non-accrual/suspended loans excluded from interest accrual, so that income is not recognized on impaired loans.
6. **6.** As an Operations Analyst, I want a loan that is paid off or closed mid-day to stop accruing further interest, so that the receivable is never overstated.
7. **7.** As the Batch/EOD process, I want a self-check that every active, accrual-eligible loan has a posted accrual entry before close, so that a partial or failed batch run is caught the same day.

Acceptance Criteria & Traceability

`REQ-CB-ACCR-001` — Daily Accrual Run Scope

Given the set of loan accounts in “Disbursed”/“Active” status as of start of business day

When the Batch/EOD accrual job runs

Then

- every account in that set is evaluated for accrual
- accounts not in “Active” status (e.g., “Pending Disbursement,” “Closed,” “Non-Accrual”) are excluded and logged as excluded, not silently skipped

Ledger-integrity constraint: *Exclusions must be explicit and logged — silent omission is indistinguishable from a batch defect.*

`REQ-CB-ACCR-002` — Balanced GL Posting for Accrual

Given an active loan account eligible for accrual with a calculated daily interest amount X

When the Posting Engine generates the accrual entry

Then

- the entry debits Interest Receivable and credits Interest Income for exactly X
- the sum of debits equals the sum of credits in the same entry
- the entry is immutable — a miscalculated accrual is corrected via a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry and immutability, identical requirement class to disbursement and repayment posting.*

`REQ-CB-ACCR-003` — Idempotent Accrual Posting

Given a loan account for which an accrual entry has already been posted for business date D

When the accrual batch is re-run for business date D (e.g., after a batch failure and restart)

Then

- no second accrual entry is posted for that loan for date D
- the batch run is safe to retry in full without producing duplicate income recognition

Ledger-integrity constraint: *Idempotency keyed on (loan account ID, business date) — critical for safe batch restart.*

`REQ-CB-ACCR-004` — Deterministic Accrual Calculation

Given a loan account with contractual annual interest rate R, day-count convention C (e.g., Actual/365), and start-of-day outstanding principal B

When the daily accrual amount is calculated

Then

- the calculation uses exactly R, C, and B as of start of day D — not intraday principal changes from same-day disbursements or repayments
- the inputs and resulting amount are recorded such that the calculation can be independently reproduced and audited

Ledger-integrity / Regulatory constraint: *Auditability — every accrual must be traceable to its rate, convention, and principal basis.*

`REQ-CB-ACCR-005` — Exclusion of Non-Accrual/Suspended Loans

Given a loan account flagged “Non-Accrual” or “Suspended” per collections/regulatory classification

When the daily accrual batch runs

Then

- no interest income is recognized for that account
- the account's non-accrual status and the date it took effect are retained for reporting, without generating a zero-value posting

Regulatory constraint: *Income recognition on impaired/non-performing loans must stop — required for regulatory reporting accuracy.*

`REQ-CB-ACCR-006` — Accrual Cutoff on Same-Day Payoff/Closure

Given a loan account that is paid off or closed during business day D, prior to the accrual batch running

When the accrual batch evaluates that account

Then

- no accrual is posted for date D if the account reached zero principal or “Closed” status before the batch cutoff
- the last accrual posted is for the final day the loan carried an outstanding balance, with no gap and no overlap

Ledger-integrity constraint: *Prevents overstatement of interest receivable on a loan with no remaining principal.*

`REQ-CB-ACCR-007` — EOD Completeness Check

```
Given the full set of accrual-eligible loan accounts for business date D
When the Batch/EOD process reaches close
Then
  • every eligible account has exactly one posted accrual entry for date D
  • any eligible account missing an entry, or any account with more than one entry,
is raised as an exception before the ledger is closed for the day
```

Ledger-integrity constraint: *Reconciliation is detective — it verifies completeness, it does not itself post entries.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
AR-7	A hypothetical ask to let an Operations Analyst manually key in an “estimated” accrued interest figure when the batch is delayed, so payoff quotes can still be issued	Direct balance mutation outside the Posting Engine. Accrued interest may only exist as a result of a posted journal entry — no manual override path.
AR-8	A hypothetical ask to auto-collect accrued interest via a new direct-debit rail not currently integrated in PaymentAdapter	New payment rail. Accrual only recognizes income; any new collection mechanism is a separate adapter-level decision requiring review.
AR-9	A hypothetical ask to net accrued interest directly against a customer's linked deposit/savings account balance	Scope beyond the six modules. Deposit accounts are out of scope; any netting must occur through Payment Execution against an external account, not a direct internal transfer to a deposit ledger this system doesn't own.

Open Questions

- What day-count convention(s) must be supported — single fixed convention system-wide, or configurable per loan product?

- Is there a defined non-accrual trigger (e.g., days-past-due threshold) this system should enforce, or is “Non-Accrual” status always set upstream by a collections system and merely respected here?

4 Loan Repayment Processing [CB-REPAY]

> **Goal:** Enable the system to accept a borrower repayment via PaymentAdapter, apply it against the correct loan account through a deterministic waterfall (fees, interest, principal), and post a single balanced, immutable journal entry per payment — so that every repayment is reflected in the loan subledger and GL with no ambiguity about how funds were allocated.

User Stories

1. **1.** As an Operations Analyst, I want an incoming payment matched to the correct loan account, so that repayments are never misapplied or left unallocated.
2. **2.** As the Posting Engine, I want each repayment applied through a fixed waterfall (fees → interest → principal), so that allocation is consistent and auditable across all payments.
3. **3.** As the Posting Engine, I want a single balanced journal entry generated per repayment, so that the GL and loan subledger stay in agreement.
4. **4.** As an Operations Analyst, I want repayment posting to be idempotent, so that a retried payment notification never double-applies funds.
5. **5.** As a Customer Service Rep, I want the repayment logged as a CRM interaction, so that customer contact history reflects payment activity without exposing ledger detail.
6. **6.** As a Batch/EOD process, I want unmatched or unallocated payments surfaced as exceptions at end-of-day, so that no cash sits unapplied past the business day.
7. **7.** As an Operations Analyst, I want a misapplied or returned repayment reversed via an offsetting entry, so that correction never alters posted history.

Acceptance Criteria & Traceability

`REQ-CB-REPAY-001` — Match Incoming Payment to Loan Account

```
Given an incoming payment received via PaymentAdapter with a loan account reference
When the system attempts to match it to an active loan account via AccountAdapter
Then
  • the payment is linked to exactly one loan account
  • a payment with no matching or an ambiguous match is routed to an exception queue rather than posted
```

Ledger-integrity constraint: *No posting occurs until a single, unambiguous account match is confirmed — prevents misapplied balances.*

`REQ-CB-REPAY-002` — Waterfall Allocation

Given a matched repayment of amount P against a loan account with outstanding fees F, accrued interest I, and principal balance B

When the Posting Engine allocates the payment

Then

- amounts are applied in fixed order: fees first (up to F), then interest (up to I), then principal (up to remaining P)
- any amount exceeding the total outstanding balance (F + I + B) is routed to a suspense/overpayment handling flow rather than applied to principal beyond payoff

Ledger-integrity constraint: *Allocation order is deterministic and must be identical for every payment — no discretionary application.*

`REQ-CB-REPAY-003` — Balanced GL Posting for Repayment

Given a completed waterfall allocation for a repayment

When the Posting Engine generates the journal entry

Then

- the entry debits Cash/Nostro and credits Fee Income, Interest Income, and Loan Receivable in the allocated amounts
- the sum of debits equals the sum of credits in the same entry
- the entry is immutable — corrections require a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry across up to three credit lines (fees/interest/principal) plus immutability.*

`REQ-CB-REPAY-004` — Idempotent Repayment Posting

Given a repayment notification with a unique payment reference ID

When the same payment reference is received more than once (e.g., due to a retry from the payment rail)

Then

- only the first occurrence results in a posted journal entry

- subsequent occurrences return the original posting result without creating a duplicate entry or reapplying the waterfall

Ledger-integrity constraint: *Idempotency keyed on payment reference ID, enforced before waterfall allocation runs.*

`REQ-CB-REPAY-005` — CRM Logging of Repayment Event

Given a successfully posted repayment

When the event completes

Then

- a CRM interaction record is created via CRMAdapter noting the payment date and loan account reference

- the CRM record contains no balance, allocation breakdown, or GL account detail

Note: *CRM exception applies — reference-only, no balance data, consistent with CRM never touching a balance.*

`REQ-CB-REPAY-006` — EOD Exception Surfacing for Unallocated Payments

Given all payments received during the business day

When the Batch/EOD process runs

Then

- every payment is in one of two states: “Posted” (fully allocated) or “Exception” (unmatched, ambiguous, or suspense)

- no payment remains in an unresolved intermediate state past EOD without appearing on the exception report

Ledger-integrity constraint: *Reconciliation is detective, not corrective — EOD flags but does not itself post or allocate.*

`REQ-CB-REPAY-007` — Reversal of Misapplied/Returned Repayment

Given a repayment that was posted and later confirmed as returned (e.g., NSF) or misapplied to the wrong loan account

```
When Operations confirms the correction
```

```
Then
```

```
  • a new reversing journal entry is posted, undoing the original
fee/interest/principal allocation in full
```

```
  • the original journal entry remains unaltered and fully traceable
```

```
  • if misapplied, a separate corrected entry is posted against the correct loan
account following the standard waterfall
```

Ledger-integrity constraint: *Immutability — reversal and correction are both new entries; nothing is edited in place.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
AR-4	A hypothetical ask to let a customer service rep manually “bump” a loan's principal balance down when a payment is confirmed verbally but not yet cleared	Direct balance mutation outside the Posting Engine. Balance may only move via a posted journal entry — verbal/unconfirmed funds cannot pre-emptively adjust a balance.
AR-5	A hypothetical ask to accept repayments via a new real-time push-to-card rail not currently supported	New payment rail. Adapter-level change requiring architecture review, not a BA-scoped requirement.
AR-6	A hypothetical ask to let overpayments automatically open an interest-bearing savings sub-account for the excess	Scope beyond the six modules. Deposit-type products are out of scope; overpayment handling should remain a suspense/refund flow within Loan Subledger + Posting Engine only.

Open Questions

- Should partial waterfall satisfaction (e.g., payment covers fees + interest but not full principal-due) still post as a single entry, or split by allocation tier?
- Is there a materiality threshold below which an overpayment is refunded automatically vs. held in suspense pending borrower instruction?

5 Delinquency & Past-Due Fee Assessment [CB-DELINQ]

> **Goal:** Enable the Batch/EOD process to identify loan accounts with a missed or partial scheduled payment as of each day's cutoff, assess any contractually-defined late fee via a balanced journal entry, and update the account's days-past-due (DPD) bucket — automatically triggering the Non-Accrual exclusion once a delinquency threshold is crossed — so that past-due loans are consistently flagged, fee-assessed, and reported with no manual daily review required.

User Stories

1. **1.** As the Batch/EOD process, I want to identify loans with a missed or partial scheduled payment as of a cutoff date, so that delinquency is tracked consistently across all accounts.
2. **2.** As the Posting Engine, I want a late fee assessed as a balanced journal entry once a missed payment passes the grace period, so that the fee is reflected in both the GL and the loan subledger.
3. **3.** As the System, I want each loan's days-past-due (DPD) bucket updated daily, so that delinquency status is always current for reporting and collections.
4. **4.** As an Operations Analyst, I want late fee assessment to be idempotent per missed payment cycle, so that a loan is never fee-assessed twice for the same missed installment.
5. **5.** As a Compliance Officer, I want a loan crossing a defined DPD threshold automatically flagged Non-Accrual, so that the accrual exclusion applies without manual intervention.
6. **6.** As a Customer Service Rep, I want delinquency status changes logged as a CRM interaction, so that collections/contact history reflects status without exposing ledger detail.
7. **7.** As an Operations Analyst, I want to waive an assessed late fee through a reversal entry, so that a legitimate exception (e.g., goodwill waiver) never leaves the ledger inconsistent with the customer-facing decision.
8. **8.** As the Batch/EOD process, I want fee assessments reconciled against DPD bucket changes, so that a fee is never posted without a corresponding status update, or vice versa.

Acceptance Criteria & Traceability

`REQ-CB-DELINQ-001` — Identify Missed/Partial Scheduled Payments

```
Given a loan account with a contractual payment schedule and a cutoff date D
```

```
When the Batch/EOD process evaluates the account as of D
```

```
Then
```

```
  • the account is flagged “past due” if the cumulative scheduled amount due through
D exceeds cumulative amounts posted via repayment processing
```

- an account current on its schedule is not flagged, even if a payment was received late but within the same cycle

Ledger-integrity constraint: *Delinquency determination reads only posted repayment entries — never an unconfirmed or pending payment.*

`REQ-CB-DELINQ-002` — Balanced GL Posting for Late Fee Assessment

Given a loan account past due beyond its contractually defined grace period

When the Posting Engine assesses the late fee

Then

- the entry debits Fee Receivable and credits Fee Income for the contractual late fee amount
- the sum of debits equals the sum of credits in the same entry
- the entry is immutable — a fee assessed in error is corrected via a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry and immutability, consistent with all prior posting requirements.*

`REQ-CB-DELINQ-003` — Daily DPD Bucket Update

Given a loan account with an existing DPD count

When the Batch/EOD process runs for date D

Then

- the DPD count increments by one calendar day if the account remains past due, or resets to zero if the account has been brought current
- the account's DPD bucket (e.g., Current, 1-29, 30-59, 60-89, 90+) is updated to reflect the new count

Ledger-integrity constraint: *DPD bucket is a status attribute, not a balance — this requirement itself posts nothing.*

`REQ-CB-DELINQ-004` — Idempotent Fee Assessment

Given a missed scheduled payment for a specific due date, uniquely identified per installment

When the late-fee assessment batch runs or is retried for that installment

Then

- only one fee assessment entry is ever posted per missed installment
- a retry or re-run does not produce a duplicate fee for the same missed due date

Ledger-integrity constraint: *Idempotency keyed on (loan account ID, scheduled due date).*

`REQ-CB-DELINQ-005` — Automatic Non-Accrual Trigger

Given a loan account whose DPD count crosses the defined non-accrual threshold (e.g., 90 days)

When the Batch/EOD process updates the DPD bucket

Then

- the account is automatically flagged “Non-Accrual” effective that date
- the accrual exclusion applies from that date forward without requiring a separate manual action

Regulatory constraint: *Consistent, threshold-driven non-accrual classification is required for accurate income recognition and regulatory reporting.*

`REQ-CB-DELINQ-006` — CRM Logging of Delinquency Status Change

Given a loan account whose DPD bucket changes (new delinquency, cure, or bucket escalation)

When the status change is finalized

Then

- a CRM interaction record is created via CRMAdapter noting the date and new status
- the CRM record contains no balance, fee amount, or GL account detail

Note: *CRM exception applies, consistent with all prior epics.*

`REQ-CB-DELINQ-007` — Fee Waiver via Reversal Entry

Given a previously posted late fee assessment

When an authorized Operations Analyst approves a fee waiver

Then

- a new reversing journal entry is posted (debit Fee Income, credit Fee Receivable) for the original fee amount
- the original assessment entry remains unaltered and fully traceable
- the waiver requires an authorization record (who approved, and why) retained alongside the reversal

Ledger-integrity constraint: *Immutability — waiver is a new entry, never an edit or deletion of the original assessment.*

`REQ-CB-DELINQ-008` — EOD Consistency Check: Fees vs. DPD Changes

Given all late fee assessments and DPD bucket changes processed during business day D

When the Batch/EOD process reaches close

Then

- every newly past-due account that crossed its grace period has a corresponding posted fee assessment, and every posted fee assessment maps to an account that is confirmed past due
- any mismatch in either direction is raised as an exception before the ledger is closed for the day

Ledger-integrity constraint: *Detective control mirroring the completeness-check pattern used elsewhere.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
`AR-16`	A hypothetical ask to let a Customer Service Rep waive a late fee by simply deleting the fee line from the customer's statement view, without a posted reversal	Direct balance mutation outside the Posting Engine. Every waiver must be a posted offsetting entry with an authorization record, never a display-layer or direct deletion.

AR-17	A hypothetical ask to automatically attempt repayment collection via a new push-to-card or instant-debit rail once an account becomes delinquent	New payment rail. Any new collection mechanism requires architecture review, regardless of the business justification.
AR-18	A hypothetical ask to auto-refer accounts crossing 90+ DPD to a third-party collections agency system with a live case-status feed	Scope beyond the six modules. A collections-agency integration is a new external system relationship, not one of the five existing adapters.

Open Questions

- Is the late fee a flat amount or a percentage of the missed installment, and does it vary by product — does this system need to store that as a per-loan contractual term, or is it a single system-wide constant?
- Does a single missed payment generate one fee per cycle, or can multiple consecutive missed cycles each generate their own fee?

6 Early Settlement / Loan Payoff [CB-PAYOFF]

> **Goal:** Enable the system to generate an accurate, time-bound payoff quote (outstanding principal + accrued interest through a stated good-through date + applicable fees), accept a payoff payment against that quote, post a single balanced journal entry that fully zeroes the loan's principal and accrued interest, and transition the account to “Closed” only once that entry is confirmed — so a borrower can settle a loan early with the ledger reflecting exact, reconciled closure and zero residual balance or accrual drift.

User Stories

1. **1.** As a Loan Officer/Customer Service Rep, I want to request a payoff quote for a loan account, so that I can tell the borrower the exact amount due to close the loan as of a given date.
2. **2.** As the System, I want the payoff quote to include outstanding principal, accrued interest through the quote's good-through date, and any applicable fees, so that the borrower is neither under- nor over-charged.
3. **3.** As the System, I want a payoff quote to carry a defined validity window, so that a stale quote can't be used to underpay after further interest has accrued.
4. **4.** As the Posting Engine, I want a payoff payment to post as a single balanced journal entry that zeroes principal and accrued interest, so that the loan subledger reflects full settlement with no residual balance.
5. **5.** As an Operations Analyst, I want payoff posting to be idempotent, so that a retried payoff payment notification never zeroes the balance twice or posts twice.
6. **6.** As the System, I want the loan account transitioned to “Closed” status only after the payoff entry is confirmed, so that the account is never closed against an unconfirmed payment.
7. **7.** As a Customer Service Rep, I want the payoff event logged as a CRM interaction, so that customer history shows the loan was settled without exposing ledger detail.
8. **8.** As the Batch/EOD process, I want every account closed via payoff checked for a true zero balance, so that any residual cent or unreconciled amount is caught before ledger close.

Acceptance Criteria & Traceability

`REQ-CB-PAYOFF-001` — Generate Payoff Quote

```
Given an active loan account with outstanding principal B and accrued interest I as of the most recent accrual
```

```
When a payoff quote is requested for a stated good-through date G
```

```
Then
```

```
  • the quote returns B, plus interest projected from the last accrual date through G, plus any applicable outstanding fees
```

- the quote is a read-only calculation — no journal entry or balance change results from generating a quote

Ledger-integrity constraint: *Quote generation must not post — a calculation is not a balance event.*

`REQ-CB-PAYOFF-002` — Quote Completeness

Given a payoff quote calculation

When the quote is generated

Then

- it itemizes principal, projected interest, and fees separately, along with the total amount due

- the components sum exactly to the quoted total, with no unexplained rounding beyond the loan's defined rounding rule

Regulatory constraint: *Itemized disclosure — borrowers are typically entitled to see how a payoff total was derived, not just a lump sum.*

`REQ-CB-PAYOFF-003` — Quote Validity Window

Given a payoff quote with good-through date G

When a payoff payment is received on or before G

Then

- the quoted total is honored as-is

- if a payoff payment is received after G, the quote is rejected as expired and a new quote must be generated reflecting additional accrued interest

Ledger-integrity constraint: *Prevents underpayment against stale interest projections — protects the accrual chain.*

`REQ-CB-PAYOFF-004` — Balanced GL Posting for Payoff

Given a payoff payment received that matches a valid, unexpired quote total exactly

When the Posting Engine generates the payoff journal entry

Then

- the entry debits Cash/Nostro and credits Loan Receivable (for B) and Interest Income (for I), zeroing both principal and accrued interest balances to exactly zero

- the sum of debits equals the sum of credits in the same entry

- the entry is immutable — any later-discovered discrepancy is corrected via a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry and immutability, consistent with all prior posting requirements.*

`REQ-CB-PAYOFF-005` — Idempotent Payoff Posting

Given a payoff payment with a unique payment reference ID

When the same payment reference is received more than once

Then

- only the first occurrence results in a posted payoff entry and account closure

- subsequent occurrences return the original result without re-zeroing the balance or re-closing the account

Ledger-integrity constraint: *Idempotency keyed on payment reference ID.*

`REQ-CB-PAYOFF-006` — Account Closure Gated on Confirmed Posting

Given a payoff payment received against a loan account

When the payoff journal entry is confirmed posted with a resulting zero balance

Then

- the loan account transitions to “Closed” status

- the account remains “Active” if the payoff entry fails to post or the resulting balance is not exactly zero

Ledger-integrity constraint: *Status transition is a downstream effect of a confirmed posting, never a precondition assumed in advance.*

`REQ-CB-PAYOFF-007` — Mismatched Payoff Amount Handling

Given a payoff payment that does not match the valid quote total exactly

When the payment is received

Then

- a payment less than the quote total does not close the account — it is applied per the standard repayment waterfall and the account remains open with a recalculated payoff quote required
- a payment greater than the quote total posts the payoff entry for the exact quoted amount and routes the excess to suspense/overpayment handling, not to the payoff journal entry

Ledger-integrity constraint: *Keeps the payoff entry itself always exactly balanced to the quote.*

`REQ-CB-PAYOFF-008` — CRM Logging of Payoff Event

Given a loan account successfully closed via payoff

When closure completes

Then

- a CRM interaction record is created via CRMAdapter noting the payoff date and loan account reference
- the CRM record contains no balance, quote breakdown, or GL account detail

Note: *CRM exception applies, consistent with all prior epics.*

`REQ-CB-PAYOFF-009` — EOD Zero-Balance Check on Closed Accounts

Given all loan accounts transitioned to “Closed” status via payoff during business day D

When the Batch/EOD process runs

Then

- every such account is confirmed to carry an exact zero principal and zero accrued-interest balance

```
  • any closed account with a non-zero residual balance is raised as an exception
before the ledger is closed for the day
```

Ledger-integrity constraint: *Detective control mirroring the completeness-check pattern used elsewhere.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged
AR-13	A hypothetical ask to let Operations manually “write off” a sub-$1 residual balance on a closed account by deleting it, rather than posting an entry	Direct balance mutation outside the Posting Engine. Even trivial residuals must clear via a posted (write-off) journal entry — no balance may be edited or deleted directly, regardless of materiality.
AR-14	A hypothetical ask to accept payoff payments via a new wire-transfer or crypto rail not currently integrated in PaymentAdapter	New payment rail. Requires architecture review before being scoped as a requirement.
AR-15	A hypothetical ask to auto-generate and email a “certificate of loan satisfaction” PDF to the borrower upon closure	Scope beyond the six modules. Document generation/correspondence is not one of the six in-scope modules — this belongs to a different solution, even though it's a plausible adjacent ask.

Open Questions

- Does the “recalculated quote required” step on underpayment need its own sub-flow, or does it simply loop back to quote generation?
- Is there a materiality threshold for the EOD zero-balance check — e.g., is a $0.01 residual due to rounding auto-written-off, or always an exception requiring human review?

7 Loan Charge-off & Write-off [CB-CHARGEOFF]

> **Goal:** Enable the system, upon a documented charge-off decision made outside this system per credit policy, to post a balanced journal entry that removes uncollectible principal and accrued interest from active receivables into the allowance/reserve for loan losses, transition the loan account to “Charged-Off” status, and preserve the full original balance history for recovery tracking and regulatory reporting — so that non-performing loans are removed from active receivables without ever deleting or altering the ledger history that produced them. The credit-policy decision to charge off a loan is explicitly out of scope; this epic begins only once that decision is received.

User Stories

1. **1.** As a Collections Manager, I want to record a charge-off decision against a severely delinquent loan account, so that it can be removed from active receivables per credit policy.
2. **2.** As the Posting Engine, I want a charge-off to post as a single balanced journal entry against the allowance for loan losses, so that the GL and subledger reflect the write-off without altering prior history.
3. **3.** As the System, I want the loan account transitioned to “Charged-Off” status only after the write-off entry is confirmed, so that status and balance are always consistent.
4. **4.** As an Operations Analyst, I want charge-off posting to be idempotent per loan account, so that a retried charge-off request never double-writes-off the same balance.
5. **5.** As the System, I want a charged-off loan excluded from further interest accrual and delinquency fee assessment, so that no further income or fees are recognized on an account already deemed uncollectible.
6. **6.** As a Collections Manager, I want to record a recovery payment against a charged-off loan, so that partial or full recoveries are reflected in the ledger without reopening the original charge-off entry.
7. **7.** As a Customer Service Rep, I want the charge-off event logged as a CRM interaction, so that customer contact history reflects the status change without exposing ledger detail.
8. **8.** As the Batch/EOD process, I want charged-off accounts reconciled to confirm the allowance/reserve balance matches the sum of all charged-off principal and interest, so that the reserve is never under- or over-stated.

Acceptance Criteria & Traceability

`REQ-CB-CHARGEOFF-001` — Record Charge-off Decision

```
Given a loan account meeting documented charge-off criteria, with outstanding
principal B and accrued interest I, and an approved credit-policy charge-off decision
```

```
When the charge-off is recorded via AccountAdapter
```

```
Then
```

- the loan account is placed in “Pending Charge-off” state referencing the approved decision

- no balance is altered until the Posting Engine confirms a balanced write-off entry

Note: *The credit-policy decision that determines charge-off eligibility is out of scope — this requirement only ingests an already-approved decision, consistent with how booking ingests an approval decision.*

`REQ-CB-CHARGEOFF-002` — Balanced GL Posting for Charge-off

Given a “Pending Charge-off” loan account with outstanding principal B and accrued interest I

When the Posting Engine generates the charge-off entry

Then

- the entry debits Allowance for Loan Losses for B and I, and credits Loan Receivable and Interest Receivable for B and I respectively

- the sum of debits equals the sum of credits in the same entry

- the entry is immutable — corrections require a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry and immutability, consistent with all prior posting requirements.*

`REQ-CB-CHARGEOFF-003` — Status Transition Gated on Confirmed Posting

Given a “Pending Charge-off” loan account

When the charge-off journal entry is confirmed posted

Then

- the loan account transitions to “Charged-Off” status

- the account remains in “Pending Charge-off” if the entry fails to post

Ledger-integrity constraint: *Status transition is a downstream effect of a confirmed posting, never a precondition assumed in advance — same pattern as payoff closure.*

`REQ-CB-CHARGEOFF-004` — Idempotent Charge-off Posting

Given a charge-off request with a unique charge-off reference ID

When the same reference is received more than once

Then

- only the first occurrence posts a write-off entry and transitions status
- subsequent occurrences return the original result without a duplicate posting

Ledger-integrity constraint: *Idempotency keyed on charge-off reference ID.*

`REQ-CB-CHARGEOFF-005` — Exclusion from Further Accrual and Fee Assessment

Given a loan account in “Charged-Off” status

When the daily accrual batch or delinquency fee assessment batch runs

Then

- the charged-off account is excluded from both, consistent with the existing non-accrual exclusion pattern
- the exclusion is logged, not silent

Regulatory constraint: *No income or fee recognition on an asset already deemed uncollectible and removed from active receivables.*

`REQ-CB-CHARGEOFF-006` — Recovery Payment on a Charged-Off Loan

Given a charged-off loan account and a recovery payment received via PaymentAdapter

When the recovery is posted

Then

- the Posting Engine generates a balanced entry crediting Recovery Income and debiting Cash/Nostro for the recovered amount
- the original charge-off entry remains unaltered and fully traceable
- the loan account remains “Charged-Off” — recovery does not reopen the account or reinstate an active receivable balance

Ledger-integrity constraint: *Immutability — recovery is always a new entry, never a reversal of the original charge-off.*

`REQ-CB-CHARGEOFF-007` — CRM Logging of Charge-off Event

Given a loan account transitioned to “Charged-Off”

When the transition completes

Then

- a CRM interaction record is created via CRMAdapter noting the charge-off date and loan account reference
- the CRM record contains no balance, reserve, or GL account detail

Note: *CRM exception applies, consistent with all prior epics.*

`REQ-CB-CHARGEOFF-008` — EOD Reserve Reconciliation

Given all charge-off entries posted through business day D

When the Batch/EOD process runs

Then

- the sum of amounts posted to the Allowance for Loan Losses from charge-offs equals the sum of principal and interest removed from active loan receivables across all charged-off accounts
- any variance is raised as an exception before the ledger is closed for the day

Ledger-integrity constraint: *Detective control mirroring the completeness-check pattern used in accrual, booking, and payoff.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged

AR-19	A hypothetical ask to let Collections directly zero out a loan's balance in the subledger view the moment a charge-off decision is made, without waiting for the Posting Engine	Direct balance mutation outside the Posting Engine. Even an approved charge-off decision must resolve through a posted, balanced entry — approval authorizes the write-off, it does not itself move a balance.
AR-20	A hypothetical ask to accept recovery payments via a new third-party debt-collection payment gateway not currently integrated in PaymentAdapter	New payment rail. Any new collection/recovery mechanism requires architecture review before being scoped as a requirement, regardless of who initiates the collection.
AR-21	A hypothetical ask to automatically sell a portfolio of charged-off receivables to a third-party debt buyer, with a live settlement feed back into this system	Scope beyond the six modules. A debt-sale relationship is a new external counterparty integration and asset-disposition capability, not a natural extension of Payment Execution or Batch/EOD — it requires its own architecture review.

Open Questions

- Does a recovery payment — partial or full — ever reinstate a loan to “Active” status, or does the loan remain permanently “Charged-Off” with recoveries tracked as a separate income line regardless of amount recovered?
- Is the DPD threshold and approval chain that constitute a valid “documented charge-off decision” enforced by this system, or is that purely an upstream credit-policy input this system simply ingests and executes?

8 Loan Modification (Rate/Term Change) [CB-MOD]

> **Goal:** Enable the system to apply an approved modification (rate change, term extension, or past-due-amount capitalization) to an existing active loan account by creating a new versioned term set with an effective date — without altering the original booked terms or any previously posted journal entry — so that a loan's contractual terms can change prospectively while every historical accrual, repayment, and disbursement entry remains fully traceable to the term version in effect when it was posted. The credit decision to approve a modification is explicitly out of scope; this epic begins only once an approved modification decision is received.

User Stories

1. **1.** As an Operations Analyst, I want to apply an approved modification to an active loan account, so that contractual changes take effect without altering the original booking record.
2. **2.** As the System, I want each modification recorded as a new versioned term set with an effective date, so that historical calculations always reference the term version in effect at the time.
3. **3.** As the Posting Engine, I want a modification that changes principal (e.g., capitalizing past-due interest) to post as a balanced journal entry, so that any balance impact is fully reflected in the ledger.
4. **4.** As the System, I want a modification that only changes rate or term with no balance impact to require no journal entry, so that a status-only change doesn't create a false balance event.
5. **5.** As an Operations Analyst, I want modification application to be idempotent per approved modification reference, so that a retried request never applies the same change twice.
6. **6.** As the Batch/EOD process, I want daily accrual to use the term version effective as of each accrual date, so that interest is calculated correctly across a modification's effective-date boundary.
7. **7.** As a Customer Service Rep, I want the modification event logged as a CRM interaction, so that customer history reflects the change without exposing ledger detail.
8. **8.** As a Compliance Officer, I want the full history of term versions retained and queryable, so that any historical calculation can be reconstructed and audited.

Acceptance Criteria & Traceability

`REQ-CB-MOD-001` — Apply Approved Modification

```
Given an approved modification decision referencing an active loan account,
containing the new term(s) (rate, remaining term, and/or restructured schedule) and
an effective date E
```

```
When the modification is applied via AccountAdapter
```

```
Then
```

- a new term version is created effective E, referencing the approved modification decision
- the loan account's terms as of any date before E remain governed by the prior term version

Note: *Modification underwriting/approval is out of scope — this requirement only ingests an already-approved decision.*

`REQ-CB-MOD-002` — Versioned, Immutable Term History

Given a loan account with one or more prior term versions

When a new modification is applied

Then

- the new term version is appended with its own effective date — no prior term version is edited or deleted
- every journal entry ever posted continues to reference the term version that was in effect at its posting date, unaffected by later modifications

Ledger-integrity constraint: *Immutability — extends the original booking's term-immutability principle to a full version chain rather than a single record.*

`REQ-CB-MOD-003` — Balanced GL Posting for Balance-Impacting Modification

Given an approved modification that capitalizes past-due interest or fees into principal, increasing outstanding principal by amount X

When the Posting Engine applies the modification

Then

- the entry debits Loan Receivable and credits Interest Receivable and/or Fee Receivable for exactly X
- the sum of debits equals the sum of credits in the same entry
- the entry is immutable — corrections require a new offsetting entry, never an edit

Ledger-integrity constraint: *Balanced entry and immutability, consistent with all prior posting requirements.*

`REQ-CB-MOD-004` — No Posting for Non-Balance-Impacting Modification

Given an approved modification that changes only rate and/or remaining term with no capitalization or balance change

When the modification is applied

Then

- no journal entry is posted — only a new term version record is created
- the loan's principal and accrued-interest balances remain exactly as they were immediately before the modification

Ledger-integrity constraint: *Mirrors the booking epic's zero-balance principle — a terms-only change is not a balance event.*

`REQ-CB-MOD-005` — Idempotent Modification Application

Given an approved modification with a unique modification reference ID

When a modification-application request is received more than once for the same reference ID

Then

- only the first occurrence creates a new term version (and posts a journal entry if applicable)
- subsequent occurrences return the original result without creating a duplicate term version or duplicate posting

Ledger-integrity constraint: *Idempotency keyed on modification reference ID.*

`REQ-CB-MOD-006` — Effective-Dated Accrual Across a Modification Boundary

Given a loan account with a modification effective date E, a prior term version, and a new term version

When the daily accrual batch calculates interest for a business date D

Then

- the calculation uses the term version whose effective date is on or before D and is the most recent such version — never a version that takes effect after D

- an accrual run spanning the exact date E correctly switches to the new term version starting on E, with no gap or double-counted day

Ledger-integrity constraint: *Deterministic, date-bounded lookup — prevents both a coverage gap and an overlap in the accrual chain across a modification.*

`REQ-CB-MOD-007` — CRM Logging of Modification Event

Given a loan account with a successfully applied modification

When the modification completes

Then

- a CRM interaction record is created via CRMAdapter noting the modification date, effective date, and loan account reference
- the CRM record contains no rate, term, or GL account detail

Note: *CRM exception applies, consistent with all prior epics.*

`REQ-CB-MOD-008` — Auditable Term Version History

Given a loan account with multiple term versions across its life

When the term history is queried (e.g., for an audit or dispute)

Then

- every version is returned with its effective date, the specific terms it carried, and a reference to the approved modification decision that created it
- the full sequence reconstructs, without gaps or overlaps, the exact terms in effect on any given historical date

Regulatory constraint: *Auditability of contractual terms over time is typically required for exam and dispute-resolution purposes.*

Architecture-Review Flags

ID	Trigger	Why It's Flagged

AR-22	A hypothetical ask to let a modification directly reduce a loan's principal balance in the subledger as a goodwill gesture, without a corresponding posted entry	Direct balance mutation outside the Posting Engine. Even a modification-driven principal change (forgiveness or capitalization) must resolve through a posted, balanced entry — never a direct subledger edit.
AR-23	A hypothetical ask to let borrowers pay a modification/restructuring fee via a new buy-now-pay-later or installment-financing rail not currently integrated in PaymentAdapter	New payment rail. Any new fee-collection mechanism requires architecture review before being scoped as a requirement.
AR-24	A hypothetical ask to let a modification automatically enroll the borrower in a linked overdraft-protection deposit product	Scope beyond the six modules. Deposit products are explicitly out of scope for this system; this belongs to a different solution entirely.

Open Questions

- Is principal forgiveness (reducing principal without a capitalization offset) a distinct modification type from capitalization, and if so, does the balance-impacting posting requirement need a separate debit line (e.g., Loss on Modification) rather than being folded into the same balanced-entry pattern?
- Can a loan have more than one modification applied on the same effective date, and if so, how is version ordering/precedence determined?

Appendix A — Requirements Traceability Matrix

Requirement ID	Epic	Epic Name	Requirement Title
REQ-CB-BOOK-001	CB-BOOK	Loan Account Booking	Create Loan Account from Approved Decision
REQ-CB-BOOK-002	CB-BOOK	Loan Account Booking	Party/CIF Linkage at Booking
REQ-CB-BOOK-003	CB-BOOK	Loan Account Booking	Immutable Contractual Terms
REQ-CB-BOOK-004	CB-BOOK	Loan Account Booking	Idempotent Booking
REQ-CB-BOOK-005	CB-BOOK	Loan Account Booking	CRM Logging of Account Opening
REQ-CB-BOOK-006	CB-BOOK	Loan Account Booking	Zero Balance at Booking
REQ-CB-BOOK-007	CB-BOOK	Loan Account Booking	EOD Reconciliation of Approvals to Bookings
REQ-CB-DISB-001	CB-DISB	Loan Disbursement Processing	Initiate Disbursement Request
REQ-CB-DISB-002	CB-DISB	Loan Disbursement Processing	Party/CIF Validation Prior to Disbursement
REQ-CB-DISB-003	CB-DISB	Loan Disbursement Processing	Balanced GL Posting for Disbursement
REQ-CB-DISB-004	CB-DISB	Loan Disbursement Processing	Idempotent Payment Execution
REQ-CB-DISB-005	CB-DISB	Loan Disbursement Processing	CRM Logging of Disbursement Event
REQ-CB-DISB-006	CB-DISB	Loan Disbursement Processing	EOD Reconciliation of Disbursements
REQ-CB-DISB-007	CB-DISB	Loan Disbursement Processing	Reversal of Failed/Returned Disbursement

REQ-CB-ACCR-001	CB-ACCR	Daily Interest Accrual	Daily Accrual Run Scope
REQ-CB-ACCR-002	CB-ACCR	Daily Interest Accrual	Balanced GL Posting for Accrual
REQ-CB-ACCR-003	CB-ACCR	Daily Interest Accrual	Idempotent Accrual Posting
REQ-CB-ACCR-004	CB-ACCR	Daily Interest Accrual	Deterministic Accrual Calculation
REQ-CB-ACCR-005	CB-ACCR	Daily Interest Accrual	Exclusion of Non-Accrual/Suspended Loans
REQ-CB-ACCR-006	CB-ACCR	Daily Interest Accrual	Accrual Cutoff on Same-Day Payoff/Closure
REQ-CB-ACCR-007	CB-ACCR	Daily Interest Accrual	EOD Completeness Check
REQ-CB-REPAY-001	CB-REPAY	Loan Repayment Processing	Match Incoming Payment to Loan Account
REQ-CB-REPAY-002	CB-REPAY	Loan Repayment Processing	Waterfall Allocation
REQ-CB-REPAY-003	CB-REPAY	Loan Repayment Processing	Balanced GL Posting for Repayment
REQ-CB-REPAY-004	CB-REPAY	Loan Repayment Processing	Idempotent Repayment Posting
REQ-CB-REPAY-005	CB-REPAY	Loan Repayment Processing	CRM Logging of Repayment Event
REQ-CB-REPAY-006	CB-REPAY	Loan Repayment Processing	EOD Exception Surfacing for Unallocated Payments
REQ-CB-REPAY-007	CB-REPAY	Loan Repayment Processing	Reversal of Misapplied/Returned Repayment

REQ-CB-DELINQ-001	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Identify Missed/Partial Scheduled Payments
REQ-CB-DELINQ-002	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Balanced GL Posting for Late Fee Assessment
REQ-CB-DELINQ-003	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Daily DPD Bucket Update
REQ-CB-DELINQ-004	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Idempotent Fee Assessment
REQ-CB-DELINQ-005	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Automatic Non-Accrual Trigger
REQ-CB-DELINQ-006	CB-DELINQ	Delinquency & Past-Due Fee Assessment	CRM Logging of Delinquency Status Change
REQ-CB-DELINQ-007	CB-DELINQ	Delinquency & Past-Due Fee Assessment	Fee Waiver via Reversal Entry
REQ-CB-DELINQ-008	CB-DELINQ	Delinquency & Past-Due Fee Assessment	EOD Consistency Check: Fees vs. DPD Changes
REQ-CB-PAYOFF-001	CB-PAYOFF	Early Settlement / Loan Payoff	Generate Payoff Quote
REQ-CB-PAYOFF-002	CB-PAYOFF	Early Settlement / Loan Payoff	Quote Completeness
REQ-CB-PAYOFF-003	CB-PAYOFF	Early Settlement / Loan Payoff	Quote Validity Window
REQ-CB-PAYOFF-004	CB-PAYOFF	Early Settlement / Loan Payoff	Balanced GL Posting for Payoff
REQ-CB-PAYOFF-005	CB-PAYOFF	Early Settlement / Loan Payoff	Idempotent Payoff Posting
REQ-CB-PAYOFF-006	CB-PAYOFF	Early Settlement / Loan Payoff	Account Closure Gated on Confirmed Posting
REQ-CB-PAYOFF-007	CB-PAYOFF	Early Settlement / Loan Payoff	Mismatched Payoff Amount Handling

REQ-CB-PAYOFF-008	CB-PAYOFF	Early Settlement / Loan Payoff	CRM Logging of Payoff Event
REQ-CB-PAYOFF-009	CB-PAYOFF	Early Settlement / Loan Payoff	EOD Zero-Balance Check on Closed Accounts
REQ-CB-CHARGEOFF-001	CB-CHARGEOFF	Loan Charge-off & Write-off	Record Charge-off Decision
REQ-CB-CHARGEOFF-002	CB-CHARGEOFF	Loan Charge-off & Write-off	Balanced GL Posting for Charge-off
REQ-CB-CHARGEOFF-003	CB-CHARGEOFF	Loan Charge-off & Write-off	Status Transition Gated on Confirmed Posting
REQ-CB-CHARGEOFF-004	CB-CHARGEOFF	Loan Charge-off & Write-off	Idempotent Charge-off Posting
REQ-CB-CHARGEOFF-005	CB-CHARGEOFF	Loan Charge-off & Write-off	Exclusion from Further Accrual and Fee Assessment
REQ-CB-CHARGEOFF-006	CB-CHARGEOFF	Loan Charge-off & Write-off	Recovery Payment on a Charged-Off Loan
REQ-CB-CHARGEOFF-007	CB-CHARGEOFF	Loan Charge-off & Write-off	CRM Logging of Charge-off Event
REQ-CB-CHARGEOFF-008	CB-CHARGEOFF	Loan Charge-off & Write-off	EOD Reserve Reconciliation
REQ-CB-MOD-001	CB-MOD	Loan Modification (Rate/Term Change)	Apply Approved Modification
REQ-CB-MOD-002	CB-MOD	Loan Modification (Rate/Term Change)	Versioned, Immutable Term History
REQ-CB-MOD-003	CB-MOD	Loan Modification (Rate/Term Change)	Balanced GL Posting for Balance-Impacting Modification
REQ-CB-MOD-004	CB-MOD	Loan Modification (Rate/Term Change)	No Posting for Non-Balance-Impacting Modification

REQ-CB-MOD-005	CB-MOD	Loan Modification (Rate/Term Change)	Idempotent Modification Application
REQ-CB-MOD-006	CB-MOD	Loan Modification (Rate/Term Change)	Effective-Dated Accrual Across a Modification Boundary
REQ-CB-MOD-007	CB-MOD	Loan Modification (Rate/Term Change)	CRM Logging of Modification Event
REQ-CB-MOD-008	CB-MOD	Loan Modification (Rate/Term Change)	Auditable Term Version History

Appendix B — Consolidated Architecture-Review Flags

These are illustrative "what-if" triggers used to demonstrate the flagging rule — none are included as active requirements above. Each represents a category (direct balance mutation outside the Posting Engine, a new payment rail, or scope beyond the six in-scope modules) that requires explicit architecture review before being scoped into a future epic.

ID	Epic	Trigger	Why It's Flagged
AR-10	CB-BOOK	A hypothetical ask to pre-load an expected receivable balance at booking so pipeline reporting shows anticipated volume before funding	Direct balance mutation outside the Posting Engine. No balance may appear without a posted entry, including for reporting-convenience reasons.
AR-11	CB-BOOK	A hypothetical ask to collect an origination/booking fee via a new payment rail at account opening	New payment rail. Any fee collection mechanism not already integrated in PaymentAdapter requires architecture review before being scoped as a requirement.
AR-12	CB-BOOK	A hypothetical ask to auto-issue a linked debit card to the borrower at booking for future repayments	Scope beyond the six modules. Cards are explicitly out of scope per the system's charter.
AR-1	CB-DISB	A hypothetical variant where Ops requests a “quick balance correction” tool to directly adjust the loan subledger without a journal entry	Direct balance mutation outside the Posting Engine. No requirement may update a balance without resolving to a balanced entry through GLPostingAdapter — this must not be silently scoped in.
AR-2	CB-DISB	A hypothetical ask for disbursement via a new instant-payment/wallet rail not currently integrated in PaymentAdapter	New payment rail. Adding a rail is an adapter/architecture change, not a BA-level requirement — needs explicit review before scoping.

AR-3	CB-DISB	A hypothetical ask to let disbursement also trigger opening of a linked savings/deposit account for the borrower	Scope beyond the six modules. Deposit products are explicitly out of scope for this system; this belongs to a different solution entirely.
AR-7	CB-ACCR	A hypothetical ask to let an Operations Analyst manually key in an “estimated” accrued interest figure when the batch is delayed, so payoff quotes can still be issued	Direct balance mutation outside the Posting Engine. Accrued interest may only exist as a result of a posted journal entry — no manual override path.
AR-8	CB-ACCR	A hypothetical ask to auto-collect accrued interest via a new direct-debit rail not currently integrated in PaymentAdapter	New payment rail. Accrual only recognizes income; any new collection mechanism is a separate adapter-level decision requiring review.
AR-9	CB-ACCR	A hypothetical ask to net accrued interest directly against a customer's linked deposit/savings account balance	Scope beyond the six modules. Deposit accounts are out of scope; any netting must occur through Payment Execution against an external account, not a direct internal transfer to a deposit ledger this system doesn't own.
AR-4	CB-REPAY	A hypothetical ask to let a customer service rep manually “bump” a loan's principal balance down when a payment is confirmed verbally but not yet cleared	Direct balance mutation outside the Posting Engine. Balance may only move via a posted journal entry — verbal/unconfirmed funds cannot pre-emptively adjust a balance.

AR-5	CB-REPAY	A hypothetical ask to accept repayments via a new real-time push-to-card rail not currently supported	New payment rail. Adapter-level change requiring architecture review, not a BA-scoped requirement.
AR-6	CB-REPAY	A hypothetical ask to let overpayments automatically open an interest-bearing savings sub-account for the excess	Scope beyond the six modules. Deposit-type products are out of scope; overpayment handling should remain a suspense/refund flow within Loan Subledger + Posting Engine only.
AR-16	CB-DELINQ	A hypothetical ask to let a Customer Service Rep waive a late fee by simply deleting the fee line from the customer's statement view, without a posted reversal	Direct balance mutation outside the Posting Engine. Every waiver must be a posted offsetting entry with an authorization record, never a display-layer or direct deletion.
AR-17	CB-DELINQ	A hypothetical ask to automatically attempt repayment collection via a new push-to-card or instant-debit rail once an account becomes delinquent	New payment rail. Any new collection mechanism requires architecture review, regardless of the business justification.
AR-18	CB-DELINQ	A hypothetical ask to auto-refer accounts crossing 90+ DPD to a third-party collections agency system with a live case-status feed	Scope beyond the six modules. A collections-agency integration is a new external system relationship, not one of the five existing adapters.
AR-13	CB-PAYOFF	A hypothetical ask to let Operations manually “write off” a sub-$1 residual balance on a closed account by deleting it, rather than posting an entry	Direct balance mutation outside the Posting Engine. Even trivial residuals must clear via a posted (write-off) journal entry — no balance may be edited or deleted directly, regardless of materiality.

AR-14	CB-PAYOFF	A hypothetical ask to accept payoff payments via a new wire-transfer or crypto rail not currently integrated in PaymentAdapter	New payment rail. Requires architecture review before being scoped as a requirement.
AR-15	CB-PAYOFF	A hypothetical ask to auto-generate and email a “certificate of loan satisfaction” PDF to the borrower upon closure	Scope beyond the six modules. Document generation/correspondence is not one of the six in-scope modules — this belongs to a different solution, even though it's a plausible adjacent ask.
AR-19	CB-CHARGEOFF	A hypothetical ask to let Collections directly zero out a loan's balance in the subledger view the moment a charge-off decision is made, without waiting for the Posting Engine	Direct balance mutation outside the Posting Engine. Even an approved charge-off decision must resolve through a posted, balanced entry — approval authorizes the write-off, it does not itself move a balance.
AR-20	CB-CHARGEOFF	A hypothetical ask to accept recovery payments via a new third-party debt-collection payment gateway not currently integrated in PaymentAdapter	New payment rail. Any new collection/recovery mechanism requires architecture review before being scoped as a requirement, regardless of who initiates the collection.
AR-21	CB-CHARGEOFF	A hypothetical ask to automatically sell a portfolio of charged-off receivables to a third-party debt buyer, with a live settlement feed back into this system	Scope beyond the six modules. A debt-sale relationship is a new external counterparty integration and asset-disposition capability, not a natural extension of Payment Execution or Batch/EOD — it requires its own architecture review.

AR-22	CB-MOD	A hypothetical ask to let a modification directly reduce a loan's principal balance in the subledger as a goodwill gesture, without a corresponding posted entry	Direct balance mutation outside the Posting Engine. Even a modification-driven principal change (forgiveness or capitalization) must resolve through a posted, balanced entry — never a direct subledger edit.
AR-23	CB-MOD	A hypothetical ask to let borrowers pay a modification/restructuring fee via a new buy-now-pay-later or installment-financing rail not currently integrated in PaymentAdapter	New payment rail. Any new fee-collection mechanism requires architecture review before being scoped as a requirement.
AR-24	CB-MOD	A hypothetical ask to let a modification automatically enroll the borrower in a linked overdraft-protection deposit product	Scope beyond the six modules. Deposit products are explicitly out of scope for this system; this belongs to a different solution entirely.